Image Captioning using Convolutional Neural Networks and Long Short-Term Memory

Joshua Moffat*
San Diego State University
STAT 702: Data Minning

May 15, 2025

Abstract

Armed with little more than a laptop and RStudio, we engineered and repeatedly refined an image-captioning system that marries a lightweight `MobileNetV2` encoder to an LSTM decoder, demonstrating how state-of-the-art deep learning can now be tamed on the desktop. Generative AI guidance accelerated the debugging of shape mismatches, masking errors, and evaluation glitches, dramatically shortening troubleshooting cycles. Our best configuration achieved a SPICE score of 0.104—promising, yet only half of the target required for highly detailed, accurate captions. Realizing that goal will demand more powerful GPUs, full-set evaluation, richer validation metrics, incremental layer unfreezing, and stronger data-augmentation. As consumer hardware and generative AI continue to advance, such workflows can be executed entirely at home, broadening access to deep-learning innovation while underscoring the need for responsible, critical use of AI tools.

1 Introduction

At the intersection of computer vision and natural language processing, image captioning enables a computer to identify content in an image to produce verbal descriptions. Image captioning has many practical applications from helping visually impaired users by narrating images, improving image search with descriptive captions, and auto-generating alt text for photos on social media. Recent advances in deep learning have greatly improved image captioning by using encoder decoder models, where an image is processed through a vision model and then generates text with a language model.

A typical encoder–decoder architecture for image captioning uses a Convolutional Neural Network (CNN) to encode the image into a feature representation, and a Recurrent Neural Network (RNN) – often a Long Short-Term Memory (LSTM) network – to decode that representation into a sentence [9]. In the pioneering work "Show and Tell", Vinyals et al. (2015) demonstrated that a CNN encoder coupled with an LSTM decoder could learn to generate realistic image descriptions, given sufficient training data. This CNN–LSTM approach (sometimes called a "Neural Image Caption Generator") became a foundation for many captioning systems [11]. The CNN extracts

*Email: jmoffat7444@sdsu.edu

high-level visual features from the image, and the LSTM sequentially produces words to form a caption, effectively translating the image features into a coherent sentence [8].

Image captioning is a supervised learning problem where we have a high dimensional predictor and a structured output. The raw pixels are a very high-p predictor vector and the caption is a structured multivariate response. Deep neural networks handle the dimensionality by learning a low dimensional latent representation (encoder) before predicting each word (decoder).

2 Methods

Our project uses the Flickr8k dataset, a corpus of 8,000 photographs, each paired with five human-written captions. The images depict a variety of everyday scenes and activities, and the captions describe important objects and actions (e.g. "Two dogs playing in the grass", "A child in a red coat is jumping in puddles", etc.). We chose Flickr8k primarily because of its manageable size, it is small enough to train on a personal laptop, yet rich enough to train a meaningful captioning model.

The goal of the project is to build an end-to-end image captioning model using a CNN–LSTM architecture. On the encoder side, we employ MobileNetV2 as the CNN backbone. MobileNetV2 is a lightweight convolutional network architecture designed for efficiency on modest hardware. It uses an inverted residual structure with depthwise separable convolutions to drastically reduce parameters while maintaining good performance [1]. For the decoder, we use an LSTM network to generate captions word-by-word. The LSTM is fed with the image's feature vector (from the CNN encoder) and learns to produce a sequence of words forming a descriptive sentence. During training, the model will learn to maximize the likelihood of the ground-truth caption given the image features. We also include an embedding layer to learn vector representations of words (so that semantically similar words might cluster in embedding space). To evaluate the quality of generated captions, we will use the SPICE metric. SPICE (Semantic Propositional Image Caption Evaluation) is a newer automatic metric that focuses on the semantic content of captions. Unlike traditional metrics (e.g. BLEU, METEOR) that rely on n-gram overlap, SPICE parses captions into scene graphs (objects and their relationships) and compares these to reference captions. This makes SPICE especially suitable for judging if the model accurately captures the objects, attributes, and relationships in the image (for example, identifying who is doing what in the scene). We choose SPICE because it has been shown to correlate more strongly with human judgment of caption quality [3].

The programming workflow includes many working pieces with the main software being RStudio with R version 4.4.2. Typically captioning systems are developed in Python with libraries like TensorFlow or PyTorch. To incorporate Python and R code within RStudio we leverage the **reticulate** package. Essentially reticulate enables R to "talk" to a live Python session running in the background, allowing the use of Python libraries and code within R scripts [2]. Miniconda, is a light weight version of the full Anaconda distribution, where it facilitates the installation of Python, its environments, files, packages, and dependencies while managing where the system looks for Python and packages through environment activation and path manipulation. In our set up, Miniconda installs Python version 3.10 and TensorFlow version 2.11.0, as well as other important Python packages like numpy, pillow, tqdm, scikit-learn, matplotlib, nltk, pycocoevalcap, and spacy. SPICE is installed from the **pycocoevalcap** package and uses Java. In order for SPICE to work, a compatible Java version must be installed and its path properly linked to RStudio. Openjdk

23.0.2 was installed using Homebrew package manager for MacOS, though could also be installed directly from Oracle's website. Overall R, Python and Java environment paths must properly link to RStudio to facilitate this work flow. The programming was done on a personal laptop running MacOS 15.3.1, with 2.3 GHz 8-Core Intel Core i9, AMD Radeon Pro 5500M 4 GB, Intel UHD Graphics 630 1536 MB, and 16 GB of RAM. During the preparation of this work the author used ChatGPT in order to research relevant literature, guide environment set-up, develop and optimize Python code, debug TensorFlow and SPICE errors, and revise written analyses. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the report.

3 Results

3.1 Data Preprocessing

Before a model can learn what is happening in an image and explain the content using words, we must process the images to be a manageable size and the captions should be stripped of non meaningful characters and words. The captions are provided in a single text file where each line contains an image filename and one caption. We assigned the file paths in R and parsed this file in Python to build a caption dataset. Each image has five captions, so after parsing we have a total of 40,000 image-caption pairs (8,000 images $\times$ 5 captions each). We then split the image IDs into a training set and validation set. Following common practice, we used 6,000 images (about 75%) for training and 1,000 for validation. The remaining 1,000 images would be held-out for final testing, but since our focus is on model development, our immediate evaluations focus on the validation set. The split was stratified so that each image's captions stay within the same set to avoid information leakage. All images were resized to 224×224 pixels and scaled to a [0,1] range by dividing pixel values by 255, which is standard for CNNs like MobileNetV2. We did not perform extensive data augmentation, such as random crops or flips, due to the relatively small dataset size and the desire to keep the image content intact for captioning (flipping an image could change its semantics, e.g. making left/right or objects' positions inconsistent with captions). However, we ensured that the resizing preserved aspect ratio by padding if necessary, to avoid warping images.

The captions were cleaned and tokenized through a series of steps. All caption text was lowercased, and punctuation was removed. For example, "A boy holding a yellow kite." becomes "a boy holding a yellow kite". We also standardized whitespace and removed any non-alphanumeric characters. We added special tokens <start> and <end> to each caption to mark the beginning and end of the sequence. For instance, the above caption would be represented as "startseq a boy holding a yellow kite endseq". During training, the model will learn to predict the next word given the previous words, and it will stop when it predicts the end token. Tokenization was applied to split captions into individual words. We then built a vocabulary of all unique words in the training captions. Because Flickr8k is a relatively small corpus, it contains many rare words that only appear once or twice. To reduce noise and model complexity, we restricted the vocabulary to words that appear at least 5 times in the training set. Words occurring <5 times were considered too rare; those tokens were replaced with an <UNK> unknown token. This frequency cutoff is a simple form of feature selection that dramatically reduces the vocabulary size while retaining the most important words. After this filtering, our final vocabulary size is 2478 words (out of 8.5k unique tokens originally). Common words like "a", "the", "on", "man", "dog", "ball" are kept, whereas very unusual words are dropped. This helps the model focus on learning to predict frequent

concepts and reduces the sparsity of the word distribution. Each caption was then converted into a sequence of word indices based on this vocabulary. We also determined the maximum caption length to use for padding. In the training data, the longest caption had about thirty words. 95% of captions are within 20 words so we decided to pad or truncate sequences to a maximum length of 20 tokens (including <start> and <end>). This length covers virtually all captions in Flickr8k, except the 5% longest captions. Shorter captions were padded with a special <PAD> token at the end so that all training sequences have uniform length, which is required for batching in training.

3.2 Model Architecture

The model architecture uses **TensorFlow Keras** and is designed to bridge visual inputs and textual outputs by using **attention**. Attention allows the model to focus on different regions of the image when generating each word, enabling it to align visual features with the evolving context of the caption. This leads to more accurate and contextually relevant descriptions by dynamically weighting the most informative parts of the image. To preserve spatial information necessary for attention, global pooling is intentionally omitted from the CNN; instead, we extract spatial feature maps and use a 1×1 convolution followed by reshaping to create a sequence of image regions. These are then projected to match the dimensionality of the decoder's hidden state using a dense layer. In parallel, a global image vector is computed via average pooling and used to initialize the LSTM's hidden and cell states, ensuring the decoder begins with a holistic summary of the visual content.

For the language generation side, the model takes in partial captions, excluding the final token, as integer-encoded sequences. These are passed through an embedding layer with $mask\_zero = True$, allowing the model to learn semantic relationships between words while ignoring padding tokens. The embedded sequences are processed by an LSTM decoder that outputs a sequence of hidden states. At each decoding step, a Luong-style attention layer computes a context vector by attending over the visual feature sequence, enabling the model to focus on relevant image regions dynamically. The context and decoder outputs are concatenated and passed through a TimeDistributed dense layer to produce vocabulary logits.

A custom masking layer, ReMask, is then used to re-apply the input mask from the caption sequence onto the logits, ensuring padded positions do not affect the training loss. Finally, a softmax activation converts these logits into a probability distribution over the vocabulary at each time step. The model is compiled with the Adam optimizer and trained using sparse categorical cross-entropy, appropriate for integer-labeled sequences.

This architecture was chosen to balance expressiveness with efficiency. By combining spatially-aware visual encoding, global context initialization, temporal sequence modeling with LSTM, and dynamic alignment via attention, the model is well-suited for the image captioning task. It can learn both global scene understanding and fine-grained region-word correspondences, which are essential for generating fluent and contextually accurate captions.

3.3 Hyper-parameter Search

To identify a strong baseline configuration for the image captioning model, we conducted an initial hyperparameter grid search using a reduced dataset of 400 training and 200 validation examples to accelerate iteration. This first-stage search aimed to narrow down promising combinations before running more computationally intensive experiments. The grid included variations in embedding dimension ('128' and '256'), LSTM units ('256' and '512'), and learning rate ('0.001' and '0.0001').

Each model was trained for up to five epochs with early stopping and learning rate reduction callbacks to prevent overfitting and improve convergence. The top three configurations, ranked by lowest validation loss, were:
(1) '$embed\_dim = 256$', '$lstm\_units = 512$', '$lr = 0.001$' with a validation loss of 3.63;
(2) '$embed\_dim = 128$', '$lstm\_units = 512$', '$lr = 0.001$' with a validation loss of 3.73; and
(3) '$embed\_dim = 256$', '$lstm\_units = 256$', '$lr = 0.001$' with a validation loss of 3.87.
These results indicate that larger LSTM units generally led to better performance at this stage, and will guide further model tuning on the full dataset.

Building on the results of the initial coarse grid search, we conducted a second-stage hyperparameter search using the full training and validation datasets to more accurately assess model performance. This round focused on the top three configurations identified earlier, keeping the learning rate fixed at '0.001' and varying the embedding dimension and LSTM size. Each configuration was trained up to 30 epochs with early stopping patience of three (epochs of no improvement in validation loss), adaptive learning rate with patience of one, and SPICE evaluation at every epoch using a custom callback. We saved two versions of model weights for each run—one corresponding to the best validation SPICE score and one restored from early stopping. The winning configuration was determined by the highest SPICE score achieved across these checkpoints. The best-performing model used '$\boldsymbol{embed\_dim = 128}$' , '$\boldsymbol{lstm\_units = 512}$', and '$\boldsymbol{lr = 0.001}$' achieving the strongest alignment between generated captions and ground truth references on the validation set, confirming its suitability for final deployment and further fine-tuning.

3.4 Fine-Tuning

3.4.1 Callbacks

To optimize the performance of our image captioning model effectively, we employed a fine-tuning approach using several methods to prevent overfitting and enhance performance. We implemented **early stopping** to terminate training when improvements in validation performance plateaued, avoiding unnecessary epochs and conserving computational resources. Additionally, we utilized custom callbacks specifically designed to calculate the SPICE validation metric. For efficiency and computational feasibility, SPICE evaluation was performed using **greedy decoding** rather than beam search. Greedy decoding generates captions by sequentially choosing the most probable next word at each step. In contrast, beam decoding simultaneously explores multiple candidate sequences at each timestep, making it computationally intensive but potentially yielding more accurate captions. For our validation callback, greedy decoding was beneficial because it significantly reduced computation time, allowing rapid, frequent assessments of the model's performance through the SPICE metric, particularly when repeated evaluations were required during training. To further optimize runtime, SPICE metrics were computed only on a subset of images from the validation dataset. Alongside these measures, we incorporated a **Reduce Learning Rate on Plateau** (RLROP) callback, automatically adjusting the learning rate downward when the validation loss ceased to improve, ensuring more stable convergence during training.

3.4.2 Learning Rate

Think of training a neural network like trying to find the bottom of a valley while blindfolded; you are talking steps based on the slope beneath your feet. A high learning rate is like taking big steps down the hill. You might get to the bottom faster, but you also risk overshooting the lowest point.

A low learning rate is like taking tiny, careful steps. You'll move more slowly, but you're less likely to overshoot. The key is to start with larger steps to make quick progress early on, and then slow down the pace as you get closer to the minimum.

3.4.3 Train CNN Layers

With the foundational hyperparameters previously established by the grid search— $\boldsymbol{embed\_dim = 128}$, $\boldsymbol{LSTM = 512}$, $\boldsymbol{lr = 1e-3}$ —we proceeded by retraining the model using the best saved weights, identified via the highest SPICE score. Fine-tuning occurred iteratively across multiple stages to gradually enhance model performance. The initial fine-tuning stage involved training the model for 10 epochs at a reduced learning rate of 1e-5, beginning with a two-epoch warm-up phase, during which all CNN layers remained frozen. Following this warm-up, we strategically unfroze the last 20 layers of the pre-trained CNN, enabling them to adapt more specifically to our captioning task. At this point, RLROP was again utilized, permitting dynamic adjustment of the learning rate for improved convergence.

In the subsequent and final stage of fine-tuning, we repeated a similar approach by reloading the previous best saved weights. Initially, the model underwent another two-epoch warm-up at a learning rate of 1e-5 with the CNN layers refrozen. After, we unfroze only the last 10 layers of the CNN and recompiled the model with a slightly increased learning rate of 5e-5 for an additional eight epochs of training. Carefully controlling the learning rate is crucial at this stage, as overly high values may result in unstable updates, while overly low rates might prevent meaningful improvements. Thus, we deliberately increased the learning rate slightly from 1e-5 to 5e-5, anticipating that the RLROP callback would soon activate and progressively lower it; this strategy prevented the learning rate from dropping prematurely to excessively low levels. Similarly, strategic unfreezing of CNN layers enables the model to leverage the general visual features learned during initial training, while progressively allowing deeper and more task-specific feature representations to emerge. Unfreezing layers of the pre-trained CNN is like having a camera already trained to recognize general objects and adjusting the settings slightly so it focuses better on the specific types of scenes and captioning vocabulary in our dataset. This nuanced, stepwise fine-tuning approach ensures stable and meaningful incremental improvement, ultimately producing a well-optimized, high-performing captioning model.

3.5 Model Diagnostics

In our preliminary grid-search phase, we monitored both training and validation cross-entropy loss to gauge the model's early learning dynamics, see Figure 1. At the outset (epoch 0), the training loss begins around 5.65 while the validation loss is already lower at approximately 4.90. Over just four epochs, both curves descend steeply—by epoch 1 the training and validation losses have dropped to roughly 4.85 and 4.75, respectively, and by epoch 4 they converge near 4.05 and 4.00. Interestingly, the validation loss remains consistently below the training loss throughout, a pattern attributable to dropout and other regularization being inactive during evaluation and a sign that our model generalizes well to unseen images early on. The rapid, parallel decline of both curves—with no indication of divergence or overfitting—confirms that our initial hyperparameter choices ($\boldsymbol{embed\_dim = 128}$, $\boldsymbol{LSTM = 512}$, $\boldsymbol{lr = 1e-3}$) provide a stable foundation.

During the retraining phase—plotted in Figure 2 for the final ten epochs of a roughly thirty-epoch fine-tuning run—we again tracked cross-entropy loss on both the training and validation splits. The

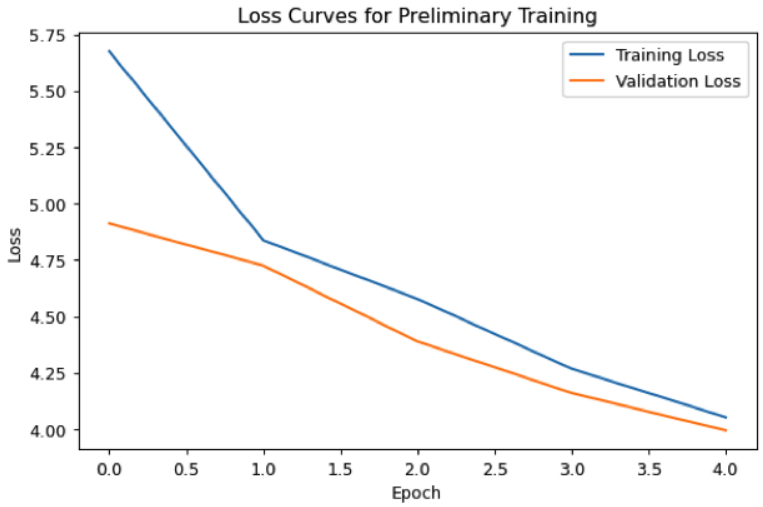

Figure 1: Loss Curves for Preliminary Grid-Search Training. The plot shows the cross-entropy loss on both the training set (blue) and validation set (orange) across the first five epochs using our initial hyperparameters. Both curves descend steeply indicating rapid early learning. The validation loss remains consistently below the training loss, a consequence of dropout and other regularization being disabled during evaluation, and reflects the model's ability to generalize to unseen images without signs of overfitting at this stage.

training loss (blue) drifts downward modestly from about 2.27 to 2.20, indicating that the model continues to adjust its weights to better fit the training captions. In stark contrast, the validation loss (orange) remains virtually flat around 2.70, with no measurable improvement across these epochs. Notably, this behavior is the inverse of what we observed during our initial grid-search training, where validation loss fell below training loss; here the training curve sits well under the validation curve, exposing a growing generalization gap. Such divergence strongly suggests that the model is beginning to overfit—learning idiosyncratic patterns in the training set that do not translate to unseen images—highlighting the need for more aggressive regularization or an earlier stopping point to preserve generalization.

In contrast to the stagnant validation loss, our SPICE-based evaluation in Figure 3 reveals that the model's semantic caption quality does continue to improve, albeit modestly, during fine-tuning. Starting from a baseline SPICE of about 0.101 at epoch 0, the metric dips slightly at epoch 2 but then climbs sharply to a peak of approximately 0.1030 by epoch 4. After that high-water mark, SPICE declines steadily, bottoming out near 0.0998 at epoch 8 before a small rebound at epoch 9. This pattern tells us that while cross-entropy loss offers a useful training signal, it does not fully capture the richness of the generated captions: the model briefly learns more semantically accurate structures around epoch 4 before over-specializing on the training set. Selecting the checkpoint at this SPICE peak ensures we preserve the finest balance between fitting the data and generating high-quality, human-like descriptions.

In our second fine-tuning cycle, initialized from the epoch-4 SPICE-optimal checkpoint, we ran ten more epochs (two warm-up epochs with all CNN layers frozen, then eight epochs after unfreezing the last ten convolutional layers at a learning rate of 5e-5). As depicted in Figure 4, the

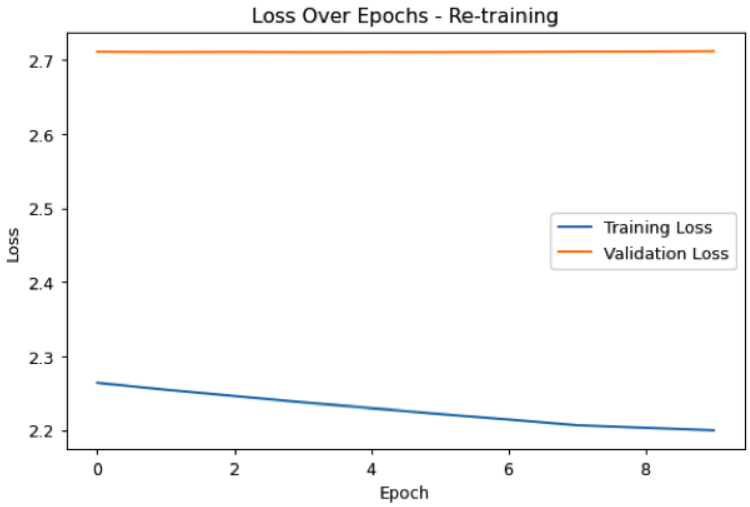

Figure 2: Loss Curves for Final Fine-Tuning Stage. During the last ten epochs of our thirty-epoch retraining, the training loss (blue) declines modestly from approximately 2.27 to 2.20, indicating continued fitting to the training captions. In contrast, the validation loss (orange) remains essentially flat at about 2.70, creating a widening gap between training and validation performance. This persistent disparity signals that the model is over-specializing on the training set's patterns without gaining further generalizable insight.

training loss (blue) steadily declines from about 2.35 down to roughly 2.08 by epoch 7, showing that the newly unfrozen layers are successfully fitting the training captions. Meanwhile, the validation loss (orange) remains essentially flat around 2.70, with even a slight upward drift. This widening gap between training and validation losses indicates that, although deeper fine-tuning sharpens model fit on known data, it does not translate into better generalization—reinforcing the need to select our final model based on peak SPICE performance and consider additional regularization to guard against overfitting.

We observed that the validation SPICE score in Figure 5 steadily improves even as the traditional loss metric has plateaued. Starting from just under 0.100 in the first epoch, SPICE dips slightly in epoch 1 but then climbs through epochs 2–5, reaching approximately 0.102, before a brief dip at epoch 6 and a final jump to a new peak of 0.104 by epoch 7. This upward trajectory, culminating in our highest semantic similarity yet, demonstrates that the model continues to refine its ability to generate more accurate, human-like captions, despite minimal changes in validation cross-entropy loss. Moving forward, we could unfreeze additional layers in small increments to further sharpen these representations; the gradual rise in SPICE indicates that each layer we expose contributes meaningful improvements to overall descriptive precision.

While our validation cross-entropy loss plateaued, and even widened relative to training loss, this token-level metric does not capture semantic relevance. In contrast, SPICE evaluates complete captions by comparing scene graphs of objects, attributes, and relations, and our SPICE scores rose steadily from about 0.100 to 0.104 during fine-tuning, revealing genuine improvements in descriptive quality. This divergence shows that a flat validation loss can mask ongoing semantic learning and underscores the value of checkpointing at SPICE peaks rather than loss minima. Furthermore,

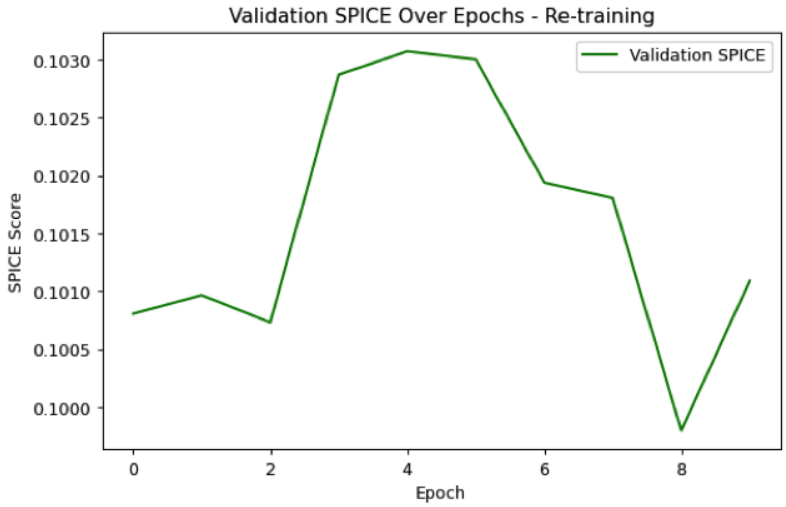

Figure 3: Validation SPICE Scores During First Fine-Tuning. The SPICE metric begins at roughly 0.101 in epoch 0, dips slightly by epoch 2, then climbs to its highest value of about 0.1030 at epoch 4—indicating that the captions produced at this point are the most semantically faithful to the ground truth. After epoch 4, SPICE steadily declines to a low near 0.0998 by epoch 8 before a modest rebound at epoch 9. This trajectory shows that even as cross-entropy loss continues to decrease on the training set, true caption quality peaks early, underscoring the value of selecting the epoch 4 checkpoint to maximize descriptive accuracy.

the upward SPICE trajectory achieved by incrementally unfreezing CNN layers confirms that each newly trainable layer enriches object and attribute recognition. By prioritizing SPICE-guided evaluation, we ensure selection of the most semantically accurate model, even when traditional loss metrics stall.

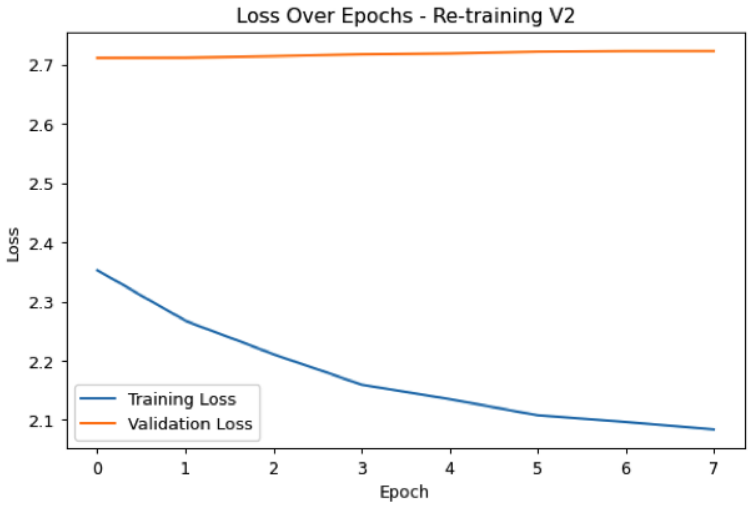

Figure 4: Training and Validation Loss During Second Fine-Tuning Cycle. After initializing from the SPICE-optimal checkpoint at epoch 4, we conducted two warm-up epochs (all CNN layers frozen) followed by eight epochs with the last ten convolutional layers unfrozen. The training loss (blue) falls steadily from about 2.35 to 2.08, demonstrating successful adaptation of these deeper features to the captioning task. In contrast, the validation loss (orange) remains essentially flat around 2.70, indicating that despite stronger fitting on the training set, and generalization to unseen images may not improve.

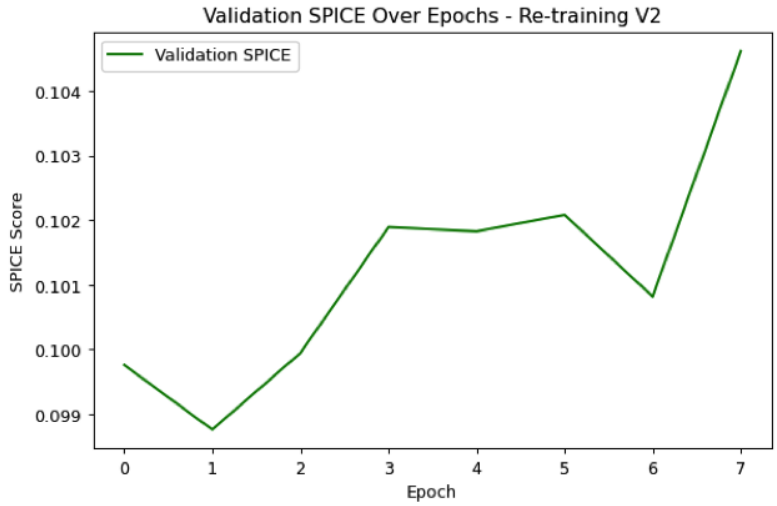

Figure 5: Validation SPICE Scores During Second Fine-Tuning Cycle. After a slight dip at epoch 1, SPICE increases roughly linearly through epochs 2–5 to about 0.102, dips marginally at epoch 6, and then surges to about 0.104 at epoch 7—indicating that progressively unfreezing deeper CNN layers yields consistent semantic improvements.

3.6 Inference and Generated Captions

For final inference, we replaced greedy decoding with **beam search** (beam width $\boldsymbol{k = 3}$) to explore multiple high-probability candidate sequences before selecting the most likely caption. Unlike the frequent SPICE evaluations during training, which demanded speed, test-time inference runs only once per image, so the extra computation is acceptable and often yields more fluent, coherent descriptions.

1. **Appendix Figure 6**
   Predicted: "a man and a woman sitting on a sidewalk"
   Ground Truth: "Two women, one sitting and one standing, near a tree."
   Interpretation: The model captures the "woman sitting" but misidentifies number and gender—likely due to dataset bias and visual symmetry—illustrating room for improvement in person-counting and gender inference.

2. **Appendix Figure 7**
   Predicted: "there is a boy in a white shirt"
   Ground Truth: "A little boy smiles and points while a man looks in the direction of the child's gaze."
   Interpretation: Beam search yields a concise, accurate core description ("boy in a white shirt") but omits the adult and the pointing action, showing that it prioritizes high-confidence object recognition over relational details.

3. **Appendix Figure 8**
   Predicted: "a man surfing in the ocean"
   Ground Truth: "A surfer surfs a wave."
   Interpretation: The model successfully identifies the surfing activity and environment, simply reframing "surfer" as "man surfing," which retains the scene's essence and demonstrates semantic flexibility.

4. **Appendix Figure 9**
   Predicted: "two dogs play in the snow"
   Ground Truth: "Two dogs play in the snow."
   Interpretation: This perfect match highlights that object-centric, unambiguous scenes are easier for the model to describe accurately.

Overall, beam search enhances caption fluency by weighing multiple hypotheses, but outputs can still stray from full ground-truth detail—especially in complex scenes requiring precise counts or relational context. Fine-tuning on more diverse person-and-action examples may help address these residual weaknesses.

4 Discussion & Conclusion

Through this project, we set out to build an end-to-end image captioning system by integrating a pre-trained CNN encoder (*MobileNetV2*) with an LSTM-based decoder, training the decoder from scratch while selectively fine-tuning the CNN layers. Our journey began with establishing a stable training pipeline in **RStudio** using the `reticulate` package which, after an initial setup

overhead, allowed us to seamlessly leverage Python libraries for model building, training, and SPICE evaluation within a familiar R environment.

Key Challenges

Along the way, we encountered and overcame several key challenges:

- **Dimension and masking issues.** Mismatched tensor shapes between the CNN output and LSTM input surfaced repeatedly, requiring careful inspection of padding and sequence lengths. We also refined our masking strategy so that padding tokens no longer contributed to the cross-entropy loss, ensuring the model focused on real caption tokens.
- **Data formatting for SPICE.** Early on, we passed lists of strings to the SPICE evaluator and had to adjust our pipeline to supply single-string captions, which resolved unexpected evaluation errors. We also discovered that SPICE scores were extremely low because we were only generating the first word of the caption during evaluation. Updating the SPICE callback to run full-sequence generation made the scores much more reflective of actual caption quality and they began to rise as the model improved.
- **Checkpointing and unfreezing.** Saving and reloading best-performing weights in HDF5 format prompted troubleshooting of file formats and naming conventions. Unfreezing the last 20 CNN layers at epoch 3 improved feature refinement but demanded tuning of learning rates to prevent catastrophic forgetting of pre-trained features.
- **Compute constraints.** Limited GPU access and RAM meant we evaluated SPICE on only a subset of the validation set, yielding a score of about 0.104—promising progress but not reflective of full-dataset performance. Scaling up to dedicated GPUs and more memory would allow a more representative evaluation and support training on larger batches.

Despite these setbacks, the SPICE score's linear improvement over epochs, even when validation cross-entropy plateaued, demonstrated that the model was still learning higher-level semantic relationships. This highlights that token-level loss and caption-level metrics capture different facets of performance: one measures next-token prediction accuracy, while the other assesses holistic scene understanding via object relations and attributes.

Key Takeaways

Looking ahead, several strategies could meaningfully enhance our image captioning model's performance. Leveraging richer resources, such as more powerful GPUs and greater memory, would allow for full-set SPICE validation, faster convergence, and the exploration of more complex architectures, including Transformer-based decoders. Expanding our evaluation metrics beyond SPICE to include BLEU, METEOR, and CIDEr would provide a more comprehensive assessment of caption quality, capturing both lexical accuracy and semantic meaning. Robustness could also be improved through data augmentation and regularization techniques, such as random cropping, color jitter, and label smoothing, which are particularly beneficial when computational resources are limited. Finally, while our transition to beam search has already improved caption quality, future work could explore beam search tuning with length penalties or diversity-promoting constraints to generate more fluent and diverse captions. Overall, this project not only solidified our technical pipeline,

from data preprocessing and model construction to training, evaluation, and debugging, but also underscored the importance of iterative problem solving in deep-learning workflows. The insights gained here lay a strong foundation for further enhancements, driving us toward more detailed, accurate, and human-like image captions.

Future Outlook & Design Statement

With the rapid evolution of consumer-grade hardware, like GPUs in laptops, larger RAM capacities, and dedicated neural-processing accelerators, individuals will soon be able to train and fine-tune Keras models entirely from their homes in minimal time. In this landscape, generative AI truly shines: instead of poring over dense documentation, learners can pose questions directly ("Why isn't my LSTM mask propagating?" "How do I reshape CNN outputs for the decoder?") and receive immediate, context-aware guidance.

Our collaboration with ChatGPT 4o, o3, and o4 exemplifies this new workflow. Instead of wrestling with API documentation or scouring forums, we could ask contextual questions and receive immediate, tailored guidance. In fact, "knowledge workers spent about a fifth of their time, or one day each work week, searching for and gathering information. If generative AI could take on such tasks, increasing the efficiency and effectiveness of the workers doing them, the benefits would be huge" [5].

Imagine a world where anyone, regardless of formal training, can leverage deep learning for local challenges—from automating crop disease detection to developing personalized language-learning tools—without wrestling with syntax or platform idiosyncrasies. As one Harvard study observed, "AI tools hold significant promise for empowering resource-constrained groups, enabling them to scale operations and enhance communications with limited time and resources" [4].

Yet, widespread access also carries responsibility. When guidance is followed uncritically, harms can ensue. As one developer ruefully recounted, "When you delegate a decision to a system you don't understand and don't check, you've surrendered agency. Worse: you've surrendered accountability" [6]. Educational research echoes this: "Students' over-reliance on AI dialogue systems ... affects their critical cognitive capabilities including decision-making, critical thinking, and analytical reasoning" [12]. As generative AI lowers technical barriers, it becomes ever more crucial to teach sound problem-solving principles and ethical reasoning alongside tool usage.

Ultimately, the barriers to entry for programming are lowering. We are constantly getting access to more powerful computers and creating higher-level abstractions, tutorials, packages, and open-source ecosystems that empower people to create and solve problems without needing deep expertise in low-level hardware/software. By combining human intuition with AI-driven support, while maintaining critical oversight and ethical reasoning, we can unlock deep learning's full promise, making it more accessible, efficient, and impactful across society.

References

[1] Mobilenetv2: Inverted residuals and linear bottlenecks. https://paperswithcode.com/method/mobilenetv2, 2023. Accessed April 29, 2025.

[2] JJ Allaire, Kevin Ushey, and Yuan Tang. Calling python from r. https://rstudio.github.io/reticulate/articles/calling_python.html, 2025. Accessed April 29, 2025.

[3] Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould. SPICE: Semantic propositional image caption evaluation. *arXiv preprint arXiv:1607.08822*, 2016. Accessed April 29, 2025.

[4] Dr. Erica Chenoweth. How ai can support democracy movements. https://ash.harvard.edu/wp-content/uploads/2025/02/How-AI-Can-Support-Democracy-Movements-Final.pdf, 2025. Harvard Kennedy School, Accessed May 13, 2025.

[5] McKinsey & Company. The economic potential of generative ai: The next productivity frontier. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier, 2023. Accessed May 13, 2025.

[6] Maxi Contieri. From helpful to harmful: How ai recommendations destroyed my os, 2025. Accessed May 13, 2025.

[7] Micah Hodosh, Peter Young, and Julia Hockenmaier. Framing image description as a ranking task: Data, models and evaluation metrics. *Journal of Artificial Intelligence Research*, 47:853–899, 2013.

[8] Shweta Pardeshi. Cnn-lstm architecture and image captioning. https://medium.com/analytics-vidhya/cnn-lstm-architecture-and-image-captioning-2351fc18e8d7, 2020. Accessed April 29, 2025.

[9] Xin Qi, Yinghuan Zhang, and Bowen Tan. Image caption generation. https://qi-xin.github.io/image%20caption%20generation.pdf, 2021. Accessed April 29, 2025.

[10] Raman Shinde. Image captioning with flickr8k dataset & bleu, 2019. Medium, May 1, 2019. Accessed April 29, 2025.

[11] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. Show and tell: A neural image caption generator. *arXiv preprint arXiv:1411.4555*, 2014. Accessed May 15, 2025.

[12] Chunpeng Zhai, Santoso Wibowo, and Lily D. Li. The effects of over-reliance on ai dialogue systems on students' cognitive abilities: a systematic review. *Smart Learning Environments*, 2024. Accessed April 29, 2025.

[13] Fengzhi Zhao, Zhezhou Yu, Tao Wang, and Yi Lv. Image captioning based on semantic scenes. *Entropy*, 2024. Accessed April 29, 2025.

Appendix

Figure 6: Predicted Caption 1

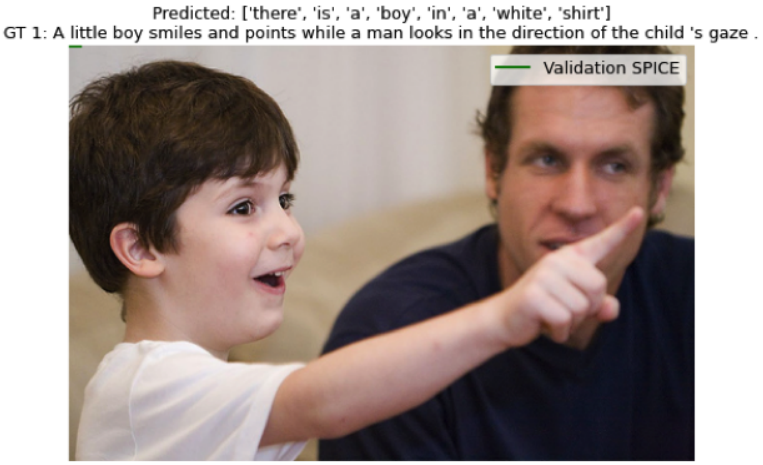

Figure 7: Predicted Caption 2

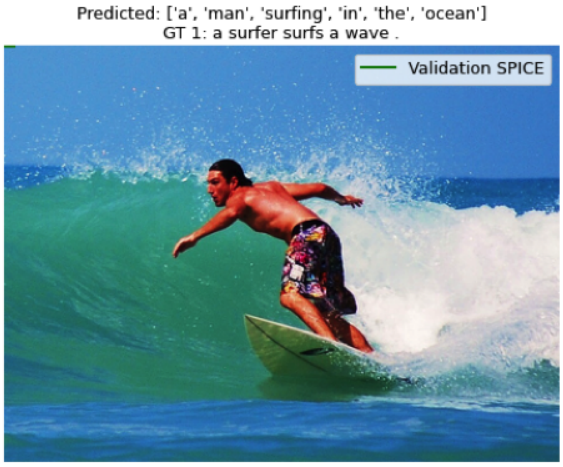

Figure 8: Predicted Caption 3

Figure 9: Predicted Caption 4